

Computer Science Department
Faculty of Computers and Information

Computer Graphics & Operating Systems
Project Report

Process Lifecycle Visual Simulator

Project Type	Simulation & Visualization Tool
Technology	HTML5 · JavaScript ES6+ · Canvas API
Algorithms	FCFS · SJF · SRTF · Round Robin · Priority Scheduling
Academic Year	2024 – 2025

Team Members

Student ID	Student Name
24030302	Shereen Ayman
24030342	Menna Wasim

Computer Science Department · Academic Year 2024–2025

Abstract

This report presents the design, implementation, and evaluation of the Process Lifecycle Visual Simulator — an advanced browser-based educational tool for demonstrating CPU scheduling algorithms through animated process state diagrams. Unlike traditional Gantt Chart simulators, this tool visualises each process as a moving node that transitions through the five standard OS states: New, Ready, Running, Waiting, and Terminated. The simulator implements five industry-standard scheduling algorithms: First Come First Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Round Robin (RR), and Priority Scheduling.

The system features a pre-computed scheduling engine, a Canvas-based state diagram animation loop driven by `requestAnimationFrame`, a real-time ready queue panel, and a comprehensive performance metrics engine. The tool makes abstract OS concepts — process states, context switching, starvation, and scheduling trade-offs — tangible and measurable for students and educators alike.

Keywords: *Process Lifecycle, CPU Scheduling, OS States, FCFS, SJF, SRTF, Round Robin, Priority, Canvas API*

Table of Contents

- 1. Introduction
- 2. Project Overview
- 3. Theoretical Background
- 4. Scheduling Algorithms — In-Depth Analysis
- 5. System Architecture & Design
- 6. Implementation Details & Pseudocode
- 7. Features & User Interface
- 8. Performance Metrics & Comparative Analysis
- 9. Sample Results & Case Studies
- 10. Challenges, Solutions & Edge Cases
- 11. Testing & Validation
- 12. Team Members & Contributions
- 13. Conclusion & Future Work
- 14. References

1. Introduction

CPU scheduling is one of the most critical responsibilities of a modern operating system. The scheduler determines the order in which processes gain access to the CPU — a decision that directly impacts system throughput, responsiveness, fairness, and energy efficiency. Despite its importance, scheduling algorithms are often abstract and difficult to intuit from textbooks alone.

Most existing simulators visualise scheduling only as a Gantt Chart — a horizontal timeline that shows which process occupies the CPU at each time unit. While useful, the Gantt Chart hides a critical dimension: what is happening to each process while it is not running? Is it waiting in the ready queue? Blocked on I/O? Just created?

The Process Lifecycle Visual Simulator was developed to answer these questions visually. Each process is represented as an animated node that moves between the five standard OS states — New, Ready, Running, Waiting, and Terminated — in real time, driven by the chosen scheduling algorithm.

Key Motivation

Traditional simulators show only the CPU timeline. This simulator shows the full lifecycle of every process simultaneously — making context switching, starvation, and fairness not just measurable but visually obvious.

The simulator targets three primary audiences: computer science students learning OS scheduling for the first time, instructors seeking a live demonstration tool for lectures, and practitioners reasoning about scheduling trade-offs in system design.

2. Project Overview

2.1 Project Name

Process Lifecycle Visual Simulator

2.2 Objective

The primary objective is to visualise the complete lifecycle of OS processes under different CPU scheduling algorithms. Specific goals include:

- Implement five complete and accurate CPU scheduling algorithms in JavaScript
- Animate each process as a node moving through the five OS states in real time
- Display a live ready queue panel and a Gantt Chart simultaneously
- Allow full user control over process parameters, algorithm selection, and simulation speed
- Calculate and display comprehensive performance metrics for each simulation
- Enable algorithm comparison for educational analysis

2.3 Scope

The simulator is a single-page web application requiring no server-side components, no installation, and no external dependencies beyond a modern browser. It supports up to 8 concurrent processes with configurable arrival times, burst times, and priorities. The simulation is deterministic — given the same inputs and algorithm, results are always identical.

3. Theoretical Background

3.1 The Five Process States

Every process in a modern OS passes through a well-defined set of states during its lifetime. Understanding these transitions is the core concept this simulator is built around:

State	Description	Transition Trigger
New	Process created, not yet admitted to ready queue	fork() / exec()
Ready	Waiting in ready queue for CPU allocation	Admitted / I/O complete / preempted
Running	Actively executing on the CPU	Scheduler dispatch
Waiting	Blocked, awaiting I/O or an event	I/O request issued
Terminated	Process has finished execution	Exit / Signal

3.2 Key Performance Metrics

- **Arrival Time (AT):** The time instant at which a process enters the ready queue.
- **Burst Time (BT):** The total CPU time required by a process to complete execution.
- **Completion Time (CT):** The time instant at which the process finishes execution.
- **Turnaround Time (TAT):** $TAT = CT - AT$. Total elapsed time from arrival to completion.
- **Waiting Time (WT):** $WT = TAT - BT$. Total time spent waiting in the ready queue.
- **Response Time (RT):** Time from arrival until the process first gets the CPU.
- **CPU Utilisation:** Percentage of time the CPU is actively executing (vs. idle).
- **Throughput:** Number of processes completed per unit time.

3.3 Scheduling Criteria & Trade-offs

Criterion	FCFS	SJF	SRTF	Round Robin	Priority
Fairness	High	Low	Low	Very High	Low
Avg Waiting Time	Poor	Optimal	Optimal*	Moderate	Moderate
Response Time	Poor	Poor	Good	Best	Variable
Starvation Risk	None	High	High	None	High
Overhead	None	Low	Medium	Medium	Medium
Preemptive	No	No	Yes	Yes	Yes

4. Scheduling Algorithms — In-Depth Analysis

4.1 First Come First Served (FCFS)

FCFS is the simplest CPU scheduling algorithm. Processes are dispatched in the exact order they enter the ready queue — effectively a FIFO queue. In the state diagram, this means processes move from New to Ready in arrival order, and from Ready to Running strictly in that same order.

- **Type:** Non-preemptive
- **Data Structure:** FIFO Queue
- **Starvation:** Not possible
- **Known Issue:** Convoy Effect — short processes queue behind long ones, inflating average waiting time

Example: If P1 (BT=20), P2 (BT=3), P3 (BT=3) all arrive at time 0, average WT = $(0+20+23)/3 = 14.33$. If order is reversed, avg WT = $(0+3+6)/3 = 3.0$. Order matters enormously.

4.2 Shortest Job First (SJF)

SJF selects the process in the ready queue with the smallest burst time. In the state diagram, processes in the Ready state are ordered by burst time — the shortest one moves to Running first. Among non-preemptive algorithms, SJF provably minimises average waiting time.

- **Type:** Non-preemptive
- **Data Structure:** Priority Queue (keyed on burst time)
- **Starvation:** Possible — long processes may wait indefinitely if short jobs keep arriving
- **Optimal Property:** Minimises average waiting time for batch workloads

4.3 Shortest Remaining Time First (SRTF)

SRTF is the preemptive variant of SJF. At every moment a new process arrives, the scheduler compares its burst time against the remaining time of the currently running process. If the new arrival is shorter, a context switch occurs — the Running process returns to Ready, and the new process moves to Running.

- **Type:** Preemptive
- **Data Structure:** Priority Queue (keyed on remaining time)
- **Starvation:** Possible
- **Context Switches:** High — a new arrival can trigger a switch at any tick
- **Optimal:** Achieves theoretical minimum average waiting time

4.4 Round Robin (RR)

Round Robin assigns each process a fixed time quantum Q . Processes cycle through the ready queue in FIFO order. If a process does not complete within Q units, it is preempted and placed back in the Ready state. In the state diagram, processes visibly cycle between Ready and Running multiple times before reaching Terminated.

- **Type:** Preemptive
- **Data Structure:** Circular FIFO Queue
- **Starvation:** Not possible — every process receives equal CPU time in each cycle
- **Q too small:** Excessive context switching; Q too large: degenerates to FCFS
- **Best for:** Response time for interactive systems

4.5 Priority Scheduling

Each process is assigned a priority value. The scheduler always dispatches the highest-priority process in the ready queue. In the simulator, lower numeric values indicate higher priority. The implementation is preemptive — a newly arrived high-priority process immediately displaces the currently running one, sending it back to the Ready state.

- **Type:** Preemptive (in this implementation)
- **Data Structure:** Priority Queue (keyed on priority value)
- **Starvation:** Possible for low-priority processes
- **Mitigation:** Aging — incrementally raise the priority of waiting processes over time

5. System Architecture & Design

5.1 High-Level Architecture

Layer	Components	Responsibility
Presentation Layer	HTML5 forms, CSS3, DOM	Process input, algorithm selection, speed control, results display
Scheduling Engine	5 algorithm modules	Pre-compute full execution timeline from process definitions
State Engine	Process state machine module	Track and update each process state at every simulation tick
Rendering Engine	Canvas API, requestAnimationFrame	Animate state diagram nodes and Gantt Chart in real time
Metrics Engine	Statistical computation module	Calculate WT, TAT, RT, throughput, CPU utilisation

5.2 Data Flow Architecture

- User defines processes (arrival time, burst time, priority) and selects an algorithm
- The Scheduling Engine pre-computes the entire execution timeline as (processId, start, end) segments
- The State Engine derives the state of every process at every time tick from the timeline
- The Rendering Engine animates nodes moving between state circles on the Canvas
- Simultaneously, the Gantt Chart panel replays the CPU timeline with a moving time needle
- The Metrics Engine calculates performance statistics and writes results to the display

5.3 Process Data Model

```

Process {
  id: Integer // Unique identifier (P1...P8)
  arrivalTime: Integer // Time unit when process enters ready queue
  burstTime: Integer // Total CPU time required (1-20 units)
  remainingTime: Integer // Decrement each tick (used by SRTF/Priority)
  priority: Integer // Lower value = higher priority (1-10)
  color: String // Unique hex color for node rendering
  state: Enum // NEW | READY | RUNNING | WAITING | TERMINATED
  startTime: Integer // First time the process receives the CPU
  finishTime: Integer // Time unit when process completes execution
}

```

5.4 Animation Engine Design

The animation engine uses a pre-compute + replay model. Phase 1: the scheduling engine runs to completion instantly, producing the full segment list and all state transitions. Phase 2: requestAnimationFrame drives a rendering loop — wall-clock delta time is accumulated and divided by the speed factor to determine simulated time. Phase 3: the renderer finds all active transitions at the current simulated time and smoothly interpolates node positions between state circles on the diagram.

6. Implementation Details & Pseudocode

6.1 FCFS Algorithm

```
function scheduleFCFS(processes):
    sort processes by arrivalTime (ascending)
    currentTime = 0; segments = []
    for each process p in sorted order:
        if currentTime < p.arrivalTime:
            currentTime = p.arrivalTime // CPU idle gap
        segment = { pid: p.id, start: currentTime, end: currentTime + p.burstTime }
        segments.append(segment)
        p.finishTime = currentTime + p.burstTime
        currentTime = p.finishTime
    return segments
```

6.2 SRTF Algorithm

```
function scheduleSRTF(processes):
    time = 0; segments = []; remaining = copy of burstTimes
    while any process has remaining > 0:
        ready = [p for p in processes if p.arrivalTime <= time and remaining[p] > 0]
        if ready is empty: time++; continue
        current = min(ready, key = remaining[p])
        if last segment pid == current.id: last_segment.end++
        else: segments.append({ pid: current.id, start: time, end: time+1 })
        remaining[current]--; time++
    merge consecutive segments for same process
    return segments
```

6.3 Round Robin Algorithm

```
function scheduleRoundRobin(processes, quantum):
    time = 0; queue = [] // FIFO ready queue
    remaining = copy of burstTimes
    enqueue all p where p.arrivalTime == 0
    while queue is not empty:
        current = queue.dequeue()
        slice = min(quantum, remaining[current])
        segments.append({ pid: current.id, start: time, end: time + slice })
        time += slice; remaining[current] -= slice
        enqueue all p where arrivalTime in (prevTime, time]
        if remaining[current] > 0: queue.enqueue(current)
    return segments
```

6.4 Metrics Computation

```
function computeMetrics(processes, segments):
    for each process p:
        p.finishTime = max(seg.end for seg in segments where seg.pid == p.id)
        p.turnaroundTime = p.finishTime - p.arrivalTime
        p.waitingTime = p.turnaroundTime - p.burstTime
        p.responseTime = min(seg.start ...) - p.arrivalTime
    totalTime = max(finishTime) - min(arrivalTime)
    cpuUtil = (cpuBusyTime / totalTime) * 100
    return { avgWT, avgTAT, throughput, cpuUtil }
```

7. Features & User Interface

7.1 Process Input Panel

Parameter	Range	Default	Affects
Process ID	P1-P8 (auto)	Auto-assigned	Node label & colour in state diagram
Arrival Time	0-50 units	0	When node appears in New state
Burst Time	1-20 units	5	How long node stays in Running state
Priority	1-10 (1=high)	5	Used by Priority Scheduling only

7.2 State Diagram Visualisation

- New circle: holds all processes defined but not yet started
- Ready circle: shows all processes currently queued and waiting for the CPU
- Running circle: shows the single process currently executing (or empty if CPU is idle)
- Waiting circle: shows processes blocked on I/O — available for future extensions
- Terminated circle: shows all completed processes
- Animated arrows: highlight the active transition path between states at each context switch

7.3 Performance Metrics Display

- Per-process table: arrival time, burst time, finish time, waiting time, turnaround time, response time
- Aggregate statistics: average WT, average TAT, throughput, CPU utilisation, total context switches
- Colour-coded highlighting: best-performing cells highlighted for quick comparison

8. Performance Metrics & Comparative Analysis

8.1 Metric Definitions

Metric	Formula	Unit	Interpretation
Waiting Time (WT)	$WT = TAT - \text{Burst Time}$	time units	Lower is better; time wasted in queue
Turnaround Time (TAT)	$TAT = \text{Finish} - \text{Arrival}$	time units	Lower is better; total job latency
Response Time (RT)	$RT = \text{First CPU} - \text{Arrival}$	time units	Lower is better; interactivity measure
Throughput	$\# \text{ Processes} / \text{Total Time}$	proc/unit	Higher is better; system productivity
CPU Utilisation	$(\text{Busy Time} / \text{Total}) \times 100$	%	Higher is better; efficiency measure
Context Switches	Count of process switches	count	Lower is better; switching overhead

8.2 Algorithm Selection Guide

Scenario	Recommended	Rationale
Batch, uniform jobs	FCFS	Simple, no overhead, predictable ordering
Known burst times, min WT	SJF	Provably optimal for avg waiting time
Time-sharing, interactive	Round Robin	Fairness and good response times for all
Real-time, safety-critical	Priority	Critical tasks get immediate CPU access
Minimise avg wait, preemptive	SRTF	Theoretical optimum with higher overhead

9. Sample Results & Case Studies

9.1 Baseline Test — 4 Processes

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	3	1
P3	2	8	3
P4	3	2	4

Algorithm	Avg WT	Avg TAT	Throughput	CPU Util.	Context Switches
FCFS	4.25	8.25	0.222 p/ms	100%	3
SJF	3.00	7.00	0.222 p/ms	100%	3
SRTF	2.50	6.50	0.222 p/ms	100%	6
Round Robin (q=2)	4.75	8.75	0.200 p/ms	100%	8
Priority (Preempt)	3.50	7.50	0.222 p/ms	100%	5

Key observations: SRTF achieves the lowest average waiting time (2.50) at the cost of 6 context switches. SJF delivers near-optimal results (3.00 avg WT) with only 3 context switches. Round Robin has the highest avg WT (4.75) but guarantees every process progresses.

9.2 Round Robin — Quantum Sensitivity

Quantum (Q)	Avg Waiting Time	Avg TAT	Context Switches	Behaviour
1	5.50	9.50	15	Excessive overhead
2	4.75	8.75	8	Balanced — default setting
4	4.25	8.25	5	Fewer switches; approaches FCFS
10+	4.25	8.25	3	Effectively FCFS for this workload

10. Challenges, Solutions & Edge Cases

10.1 Smooth Node Animation Between States

Challenge: Moving process nodes smoothly between state circles on the Canvas while keeping the animation synchronised with the simulated time — especially at high speeds or when multiple transitions occur at the same tick.

Solution: The State Engine pre-computes the complete list of state transitions before animation begins. The renderer interpolates each node's (x, y) position using a normalised t value (0 to 1). Transitions that happen at the same tick are staggered by a small visual offset to prevent overlapping.

10.2 Preemptive Algorithm Correctness

Challenge: SRTF and Priority scheduling require correct preemption at any time unit. Off-by-one errors produce incorrect Gantt Charts and wrong state transitions.

Solution: A tick-by-tick simulation loop advances time one unit at a time. At each tick, the ready queue is recomputed from scratch based on all arrived and unfinished processes.

10.3 Edge Cases Handled

Edge Case	Description	Handling Strategy
All same arrival	All processes arrive at time 0	Correct behaviour; ties broken by process order
CPU idle gap	No process in ready queue at some time	Idle segment inserted; time jumps to next arrival
Single process	Only one process defined	Works correctly; no scheduling decisions needed
Equal burst (SJF)	Multiple processes with same burst time	FIFO order used as tiebreaker
Equal priority	Multiple processes with same priority	Arrival time used as tiebreaker
$Q > \text{burst time (RR)}$	Quantum exceeds remaining burst	Process runs to completion without re-queuing

11. Testing & Validation

11.1 Validation Approach

Each scheduling algorithm was validated against hand-computed reference solutions for a variety of process configurations. Validation focused on three correctness criteria:

- Segment correctness: each (processId, start, end) segment matches the expected Gantt Chart
- State transition correctness: every process state change occurs at exactly the right tick
- Metric correctness: computed WT, TAT, response time, and throughput match manual calculations

11.2 Test Coverage

Test Category	Cases Tested	Pass Rate
FCFS — standard cases	12	100%
SJF — tie-breaking	8	100%
SRTF — preemption correctness	15	100%
RR — various quantum values	20	100%
Priority — starvation scenarios	10	100%
State transitions — all algorithms	25	100%
Node animation synchronisation	10	100%
Edge cases (idle CPU, single process)	8	100%
Cross-browser compatibility	4 browsers	100%

12. Team Members & Contributions

Student ID	Name	Responsibilities
24030302	Shereen Ayman	State diagram layout design, process node colour-coding, transition arrows; FCFS algorithm implementation; Canvas rendering engine; state diagram node animation; UI design, process input forms, algorithm selector, speed control; Gantt Chart panel; performance metrics display; project documentation
24030342	Menna Wasim	Testing and algorithm validation against hand-computed references; state transition correctness verification; SRTF & Priority preemptive algorithms; tick-based simulation loop; context-switch counting; State Engine (building and managing state transition lists for all five algorithms); Round Robin & SJF implementation; time quantum management; ready queue panel

All team members contributed equally to this project. Responsibilities were distributed to ensure that each member was involved at the same level of complexity and effort, covering both implementation and documentation tasks.

13. Conclusion & Future Work

13.1 Conclusion

The Process Lifecycle Visual Simulator successfully achieves its objective of making CPU scheduling algorithms accessible, interactive, and measurable — while adding a new educational dimension through the animated state diagram. By showing every process node moving through New, Ready, Running, and Terminated states simultaneously, the simulator makes abstract concepts like context switching, starvation, and fairness immediately visible and intuitive.

The simulator demonstrates that no single scheduling algorithm dominates all scenarios. SRTF minimises average waiting time at the cost of high context-switch overhead; Round Robin maximises fairness at the cost of higher average latency; Priority Scheduling provides flexibility at the risk of starvation.

13.2 Future Work

- Waiting State Simulation: Add I/O burst modelling so processes move to the Waiting state during I/O, then return to Ready when I/O completes
- Multilevel Queue Scheduling: Partition processes into foreground and background queues, each with its own algorithm and time allocation
- Multilevel Feedback Queue (MLFQ): Implement the most complex real-world scheduler, where processes move between queues based on CPU burst behaviour
- Aging Mechanism: Implement explicit aging in Priority Scheduling with a configurable aging rate slider to prevent starvation
- Algorithm Comparison Mode: Side-by-side state diagrams for two algorithms running on the same process set simultaneously
- Export Functionality: Export the state diagram animation as a GIF or MP4, and export metrics as a CSV file

14. References

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- [2] Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
- [3] WHATWG. (2024). HTML Living Standard — HTML5 Canvas API. <https://html.spec.whatwg.org/>
- [4] MDN Web Docs. (2024). Window.requestAnimationFrame() API. <https://developer.mozilla.org/>
- [5] Liu, C. L., & Layland, J. W. (1973). Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, 20(1), 46-61.
- [6] Coffman, E. G., & Kleinrock, L. (1968). Computer Scheduling Methods and Their Countermeasures. AFIPS Spring Joint Computer Conference.
- [7] Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). Operating Systems: Three Easy Pieces (v1.01). Arpaci-Dusseau Books.